Security at the Source: A Case Study of Shared Code Vulnerabilities in Serverless API Gateways

Sai Mani Ritish Upadhyayula,
upadhyayulasaimanir@cityuniversity.edu
Akarsh Lakshmana,
lakshmanaakarsh@cityuniversity.edu
Divakar Reddy Ravi,
ravidivakarreddy@cityuniversity.edu
Yashasree Duggiempudi,
kduggiempudiyashasre@cityuniversity.edu

School of Technology & Computing
City University of Seattle

Abstract

The modern software ecosystem increasingly relies on shared codebases, open-source libraries, and third-party integrations to accelerate development and deployment. In serverless computing environments, this reliance is amplified due to the widespread reuse of API Gateway configurations, authentication of middleware, software development kits (SDKs), and open-source dependencies across multiple microservices. While this approach improves development efficiency and software quality, it also introduces systemic security risks when vulnerable or poorly reviewed code is replicated across platforms. This case study analyzes security failures in serverless microservices that originate from insecure shared code and third-party libraries used within API Gateway–based architectures. The study examines how vulnerabilities at the source such as insecure authorization logic, outdated dependencies, and misconfigured shared components can be exploited at scale. Through a comparative analysis of serverful and serverless security models, this research evaluates how Security Development Lifecycle (SDL) practices, including secure code reviews and threat modeling, can mitigate risks associated with shared code reuse. The paper concludes by recommending development-time security controls to strengthen application-layer defenses in serverless environments.

Keywords: Serverless Computing, API Gateway Security, Shared Code, Open-Source Libraries, Software Supply Chain Security, Secure SDLC, Microservices Architecture

1. INTRODUCTION

Serverless computing has transformed modern application development by abstracting infrastructure management and allowing developers to focus solely on business logic. Cloud providers such as Amazon Web Services, Microsoft Azure, and Google Cloud Platform offer API Gateway services integrated with Function-as-a-Service (FaaS) platforms to build scalable microservices architectures. However, this architectural evolution has coincided with an increased reliance on shared open-source libraries and reusable components. While serverless systems eliminate many traditional server management risks, they amplify software supply chain exposure because application security becomes tightly coupled with the integrity of third-party dependencies.

The principle of "Security at the Source" emphasizes embedding security practices at the earliest stages of development rather than addressing vulnerabilities of post-deployment. In environments where APIs rely heavily on external packages and shared code modules, insecure dependencies can propagate rapidly across multiple systems. This paper analyzes how such vulnerabilities affect serverless API Gateways and how Secure Development Lifecycle (SDL) practices can mitigate the risks.

2. MOTIVATION

The rapid adoption of microservices and serverless computing has led to unprecedented reuse of open-source components. Research indicates that modern applications may contain hundreds of third-party dependencies, many of

which introduce transitive vulnerabilities. High-profile incidents such as dependency confusion attacks and malicious package injections have demonstrated how attackers exploit trusted repositories to infiltrate production systems.

In serverless architectures, API Gateways acts as centralized entry points, making them high-value targets. A single vulnerable dependency within request validation logic, authentication middleware, or logging modules can compromise the entire service layer. Because serverless environments automatically scale and distribute workloads, vulnerabilities can propagate faster and affect a broader attack surface compared to traditional monolithic systems. This creates a strong need for proactive, source-level security enforcement.

2.2 Problem Statement

Although serverless computing improves scalability and operational efficiency, it introduces systemic risks when shared code and third-party libraries are not properly validated. Vulnerabilities in open-source packages may remain undetected until exploited, and cloned code segments can distribute insecure logic across multiple microservices. In API Gateway architectures, flaws in authentication libraries, input validation modules, or dependency injection frameworks may enable unauthorized access, data exposure, or remote code execution.

Traditional security approaches often focus on runtime protection; however, this reactive model fails to address vulnerabilities embedded in source code dependencies. The problem addressed in this study is how shared code vulnerabilities within serverless API Gateways can be systematically identified, prioritized, and mitigated using Security at the Source principles integrated within the Secure Development Lifecycle.

1.3 Usefulness

This study contributes to both academic and practical domains by demonstrating how SDL practices can reduce supply chain vulnerabilities in serverless systems. It provides a structured threat modeling framework applicable to cloud-native architectures and offers actionable security implementation strategies such as automated dependency scanning, secure CI/CD enforcement, and least privilege access management.

For practitioners, the findings offer guidance on improving security posture in API-driven architectures. For researchers, the paper highlights the evolving nature of supply chain threats and emphasizes the importance of integrating governance, policy enforcement, and automated security validation into modern development workflows.

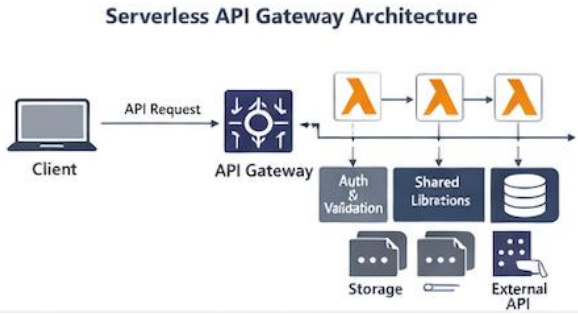

Figure 1. Typical serverless API Gateway architecture integrating backend functions and cloud services

3. LITRATURE REVIEW

Existing research highlights the increasing prevalence of software supply chain attacks in modern cloud-native systems. Studies comparing traditional serverful applications to serverless architectures show that while infrastructure risks decrease in managed environments, dependency-based vulnerabilities increase due to widespread code reuse. Professional security analyses emphasize that open-source libraries accelerate innovation but introduce risk through outdated versions, lack of maintenance, and insufficient validation.

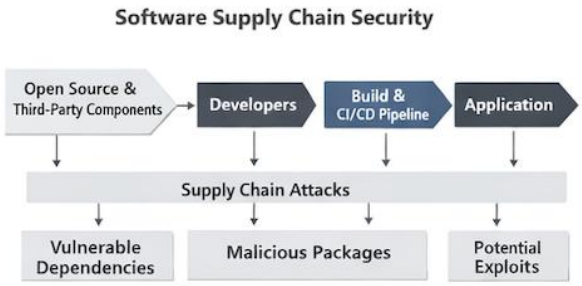

Figure 2. Software supply chain model illustrating third-party dependency integration risks (CISA, 2023).

Academic research on Secure Development Lifecycle practices demonstrates that early integration of threat modeling and automated testing significantly reduces exploitable defects. Comparisons across studies reveal that organizations implementing continuous dependency scanning and least privilege policies

experience fewer high-severity incidents. However, gaps remain in applying these controls consistently within serverless microservices environments, particularly in API Gateway security.

The transition from custom-built monolithic systems to modular serverless architectures has shifted the primary attack vector from infrastructure misconfiguration to software supply chain exploitation. This shift necessitates proactive governance and automation in security validation processes.

Vulnerability Type	Example Impact	Exploitation Vector
Dependency Confusion	Malicious code execution	Build pipeline injection
Outdated Libraries	Known CVEs exploited	Runtime invocation
Injection Flaws	Data manipulation	API payload injection
Broken Auth	Token forgery	Spoofed requests

Table 1: Common Vulnerabilities Introduced Through Shared Code Dependencies

3. Threat Modeling and Attack Surface Analysis

The analyzed system exposes RESTful APIs through a publicly accessible API Gateway. Primary attack surfaces include authentication of endpoints, request parsing mechanisms, dependency loading processes, and logging frameworks. Using the STRIDE model, spoofing threats were associated with weak token validation mechanisms; tampering risks were linked to insecure dependency injection, and denial-of-service threats targeted unprotected API endpoints.

Threat prioritization classified broken authentication and vulnerable third-party packages as high-risk due to their potential to enable lateral movement and data compromise. Information disclosure risks were associated with verbose error handling and insecure logging practices. The structured threat profile demonstrates that shared code components significantly expand the attack surface compared to custom-built isolated systems.

Threat	Likelihood	Impact	Risk Level
Dependency Confusion	High	High	Critical
Token Forgery	Medium	High	High
API DoS	High	Medium	High
Data Leakage	Medium	Medium	Medium

Table 2: Risk Prioritization of Identified Threats in the Serverless API Gateway System

4. METHODOLOGY

The case study focuses on a representative serverless architecture utilizing an API Gateway integrated with cloud-based functions and multiple open-source dependencies. The system was selected based on three criteria: reliance on third-party libraries, public API exposure, and automated CI/CD deployment pipeline integration.

Threat modeling was conducted using the STRIDE framework to categorize risks including spoofing, tampering, repudiation, information disclosure, denial of service, and privilege escalation. Attack surface analysis identified externally accessible endpoints, dependency injection points, authentication modules, and logging services as critical areas of risk.

Vulnerability identification was performed through analysis of publicly available CVE databases, OWASP documentation, and dependency scanning tools. Resources such as the OWASP Foundation guidelines and National Vulnerability Database were used to evaluate known risks in shared components.

The study applied SDL concepts (A1–A5) across development stages, including security training, threat modeling, secure coding reviews, automated dependency management, and final security validation. Security measures were evaluated based on their ability to reduce exploit likelihood and limit attack impact.

4.1 A1 Analysis – Security Training and Policy Compliance

The A1 phase evaluates whether foundational security practices and governance mechanisms were in place before development began. In the analyzed system, developers integrated several open-source packages for JWT validation, request parsing, and API rate limiting. However, there

was no documented policy requiring verification of package maintainers, dependency integrity validation, or formal security review before integration.

Security awareness training specific to supply chain risks was limited. Developers relied on package popularity metrics rather than vulnerability history when selecting dependencies. As a result, outdated libraries containing known CVEs remained in production environments.

From an SDL perspective, A1 deficiencies directly contributed to vulnerability introduction. Without formalized dependency governance policies and mandatory secure coding training, the system became exposed to risks such as dependency confusion, malicious package injection, and insecure cryptographic implementation. Strengthening A1 controls through mandatory training programs and formal dependency approval of workflows would significantly reduce source-level vulnerability insertion.

4.2 A2 Analysis – Threat Modeling and Risk Identification

Under A2, structured threat modeling was applied using the STRIDE framework to evaluate attack vectors within the serverless API Gateway architecture. The system's primary attack surfaces included publicly exposed API endpoints, authentication of middleware, CI/CD build pipelines, and dependency resolution mechanisms.

Spoofing threats were identified in token validation logic that did not strictly verify algorithm enforcement. Tampering risks emerged through potential dependency confusion attacks, where a malicious public package could override a private internal module. Repudiation risks were associated with insufficient logging configuration in the API Gateway. Information disclosure vulnerabilities were linked to verbose error responses returned by shared middleware libraries. Denial-of-service threats targeted rate-limiting misconfigurations. Elevation of privilege risks stemmed from overly permissive IAM roles attached to serverless functions.

Threat prioritization revealed that shared code vulnerabilities expanded the attack surface beyond traditional infrastructure risks. The absence of early-stage modeling allowed supply chain risks to remain unidentified until verification phases. Proper A2 execution would require systematic modeling of dependency trust boundaries and CI/CD exposure risks before implementation.

4.3 A3 Analysis – Secure Design and Implementation

The A3 phase examined whether secure design principles were applied during implementation. The case study revealed that authentication logic was duplicated across multiple serverless functions using a shared external library. This architectural decision increased the blast radius of any vulnerability discovered within the authentication module.

Additionally, dependency versions were not pinned, allowing automatic minor updates that could introduce unverified code changes into production builds. IAM permissions granted broad access to storage and logging services, violating least privilege principles.

Secure design deficiencies in A3 amplified shared code risks. Proper SDL implementation would require centralizing authentication validation at the API Gateway level, enforcing strict least privilege access policies, isolating dependencies through private repositories, and implementing version locking mechanisms. These design improvements would limit lateral movement and reduce systemic vulnerability propagation.

4.4 A4 Analysis – Verification and Security Testing

During A4, verification mechanisms were evaluated to determine whether vulnerabilities could be detected before deployment. The CI/CD pipeline lacks automated Software Composition Analysis (SCA) scanning for dependency vulnerabilities. No static analysis tools were configured to identify insecure cryptographic practices or unsafe input handling within third-party modules.

Dynamic API testing was minimal and did not simulate adversarial scenarios such as dependency injection manipulation or malformed token payloads. As a result, several vulnerabilities remained undetected until manual review.

The absence of rigorous verification under A4 significantly weakened the system's security posture. Effective implementation would require automated dependency scanning integrated into the pipeline, static application security testing

(SAST), dynamic API fuzz testing, and enforcement gates preventing deployment if high-severity vulnerabilities are detected.

Figure 3. Secure Development Lifecycle integrating security across development phases (Microsoft, 2022).

4.5 A5 Analysis – Release Governance and Continuous Monitoring

The final SDL phase assessed post-deployment governance and monitoring mechanisms. The analyzed system used centralized logging but lacked anomaly detection or automated alerting tied to suspicious API invocation patterns. Dependency updates were performed reactively rather than proactively based on vulnerability disclosures.

There was no formal vulnerability response plan outlining timelines for patching disclosed CVEs affecting integrated libraries. This governance gap increased the window of exposure after vulnerabilities became publicly known.

Proper A5 implementation would include continuous monitoring of API behavior, automated vulnerability intelligence integration, mandatory patch timelines, and security review checkpoints for dependency updates. Continuous governance ensures that Security at the Source remains effective throughout the operational lifecycle.

SDL Activity	Risk Identified	Mitigation Strategy
A1	Unvetted dependencies	Secure coding training
A2	Expanded attack surface	Structured threat modeling
A3	Over-permissive design	Least privilege enforcement
A4	Undetected CVEs	Automated SCA scanning
A5	Delayed patching	Continuous monitoring

Table 3: Mapping of SDL Activities to Identified Risks and Mitigation Controls

5. Multi-Stage RCE in Serverless Environments

The exploitation of shared code vulnerabilities in serverless API Gateway architectures often follows a structured, multi-stage execution chain rather than a single-point compromise. In modern cloud-native systems, remote code execution (RCE) rarely occurs directly from external input. Instead, adversaries leverage weaknesses in dependency management, authentication of middleware, or request processing libraries to progressively escalate privileges and execute malicious payloads. This study identifies a gated Remote Code Execution (RCE) pathway that demonstrates how shared code vulnerabilities propagate across multiple architectural layers.

The initial stage typically begins with reconnaissance and dependency enumeration. Attackers analyze publicly accessible API endpoints and inspect response headers, error messages, and exposed metadata to identify underlying frameworks and third-party libraries. In serverless systems, verbose error handling or stack traces returned by shared middleware components may inadvertently reveal version numbers of libraries. Once a vulnerable dependency is identified—particularly one associated with deserialization flaws, unsafe template rendering, or insecure cryptographic handling—the attacker transitions to exploitation attempts.

The second stage involves bypassing gateway-level controls. In a gated RCE scenario, execution is "gated" by authentication and input validation logic embedded within shared middleware. If the authentication library contains a flaw, such as improper JWT signature verification or algorithm

confusion vulnerability, the attacker may forge tokens to gain unauthorized access. Alternatively, injection vulnerabilities within request parsing libraries may allow malicious payloads to traverse validation filters. At this stage, the attacker does not yet achieve code execution but establishes unauthorized API invocation capability.

The third stage involves exploitation of backend execution contexts. In serverless architectures, API Gateways invokes cloud functions with associated IAM roles. If a shared dependency contains a deserialization of vulnerability or command injection flaw, the attacker can craft a payload that triggers arbitrary code execution within the function runtime environment. Because serverless functions often inherit permissions to access storage, databases, or messaging queues, successful RCE enables lateral movement within the cloud environment.

The "gated" nature of this RCE chain lies in the layered trust relationships. Each stage depends on bypassing prior controls: authentication validation, input sanitization, dependency resolution integrity, and IAM role restrictions. Unlike traditional monolithic server compromises, serverless RCE frequently requires chaining multiple weaknesses across shared components. However, once achieved, the automatic scalability of serverless platforms amplifies impact, potentially allowing rapid exploitation across numerous concurrent function instances.

Furthermore, CI/CD pipelines introduce an additional stage in the execution chain. Dependency confusion attacks exploit build-time resolution mechanisms by publishing malicious packages under names like private internal modules. When the build pipeline resolves dependencies automatically, the malicious package is integrated into production artifacts. This pre-deployment compromise represents a form of indirect RCE, where malicious code executes within cloud functions upon invocation without requiring runtime injection.

The analysis demonstrates that shared code vulnerabilities transform API Gateways into aggregation points of systemic risk. The multi-stage execution chain highlights that RCE in serverless systems is rarely isolated; rather, it is the culmination of failures across governance, design, and verification layers. Therefore, effective defense must interrupt the execution chain at multiple stages rather than relying on a single protective mechanism.

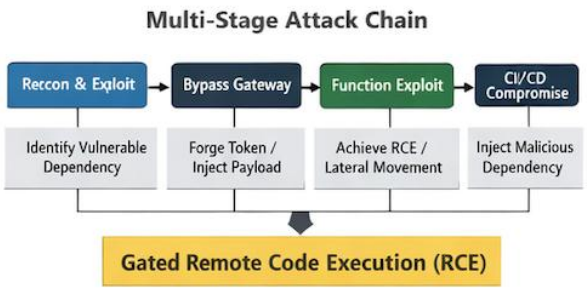

Figure 4. Multi-stage attack chain illustrating dependency-based gated remote code execution (adapted from MITRE ATT&CK, 2023).

6. SECURITY MEASURES & REMEDIES

Mitigating multi-stage execution chains in serverless API Gateways requires layered security controls integrated across the Secure Development Lifecycle. Effective remedies must address both source-level dependency risks and runtime execution constraints.

The first essential control is secure dependency governance. All third-party libraries should be version-pinned and sourced from verified repositories. Private package registries should be implemented to prevent dependency confusion attacks. Automated Software Composition Analysis (SCA) tools must be integrated into CI/CD pipelines to detect known vulnerabilities before deployment. Build processes should enforce integrity verification through checksum validation and artifact signing to prevent malicious code injection.

Authentication and authorization of hardening is critical to interrupting the gated RCE chain. API Gateways should enforce strict token validation, including algorithm enforcement and expiration verification. Multi-factor authentication and role-based access control policies should be implemented to reduce spoofing risks. Backend serverless functions must operate under least privilege IAM roles, ensuring that even if RCE occurs, lateral movement is restricted.

Input validation and output encoding mechanisms should be standardized across all functions. Centralized validation libraries maintained internally are preferable to distributed third-party implementations, reducing dependency fragmentation. Deserialization routines must avoid unsafe object reconstruction, and command execution interfaces should be strictly sandboxed.

Runtime monitoring provides an additional defensive layer. Centralized logging combined with anomaly detection algorithms can identify irregular invocation patterns or privilege escalation attempts. Rate limiting and throttling policies implemented at the API Gateway level can mitigate denial-of-service attacks and limit brute-force exploitation attempts.

Secure build and release governance further strengthens defenses. All dependency updates should require security review approval. Continuous vulnerability intelligence feeds must be integrated to trigger immediate patching workflows when new CVEs are disclosed. Regular threat modeling reviews should reassess evolving attack vectors, particularly those targeting supply chain components.

Finally, adopting a zero-trust security posture ensures that no component, including internal shared libraries, is inherently trusted. Every invocation, dependency integration, and privilege grant must be validated explicitly. By embedding these remedies across development, deployment, and operational phases, organizations can effectively disrupt the multi-stage execution chain and significantly reduce the probability and impact of gated Remote Code Execution in serverless API Gateway environments.

7. POST-RELEASE SECURITY

Maintaining security in serverless API Gateway architectures after deployment requires a continuous and adaptive approach, as new vulnerabilities in shared code and third-party dependencies emerge over time. In the analyzed case study, post-release security focuses on three critical areas: vulnerability monitoring, patch management, and incident response. Organizations must implement continuous vulnerability scanning mechanisms that monitor dependencies against publicly disclosed vulnerabilities in databases such as those maintained by the National Institute of Standards and Technology and industry frameworks like the OWASP Foundation. These scanning tools should be integrated into operational pipelines to detect newly discovered Common Vulnerabilities and Exposures (CVEs) affecting deployed serverless functions and API Gateway configurations [17].

Patch management in serverless environments presents unique challenges due to the distributed and dependency-driven nature of applications. Unlike traditional systems, where patching may involve updating operating systems or middleware, serverless systems require rapid updating of third-party libraries and redeployment of functions. Automated patch pipelines should be established to ensure that critical vulnerabilities are remediated within defined service-level timelines. Additionally, organizations must maintain version control and dependency tracking to ensure that updates do not introduce regressions or compatibility issues. Risk-based prioritization methods such as DREAD scoring can be applied to evaluate the severity of vulnerabilities based on potential damage, exploitability, and affected user scope, allowing teams to prioritize remediation effectively.

Responsible disclosure and incident response strategies are equally important in maintaining long-term security. Organizations should establish clear policies for reporting vulnerabilities discovered by internal teams or external researchers. Centralized logging and monitoring systems must be configured to detect anomalous API activity, such as repeated authentication failures or unusual invocation patterns, which may indicate exploitation attempts. Security teams should conduct regular post-deployment reviews and penetration testing exercises to validate the effectiveness of implemented controls. By combining continuous monitoring, structured patch management, and proactive incident response, organizations can ensure that security remains resilient even as new threats emerge in the software supply chain.

8. CONCLUSION & RECOMMENDATION

This study demonstrates that shared code and third-party dependencies introduce significant security risks in serverless API Gateway architectures, particularly when secure development practices are not consistently applied. The analysis across SDL activities A1 through A5 reveals that vulnerabilities often originate at early development stages due to insufficient security training, lack of dependency governance, and absence of structured threat modeling. These vulnerabilities are further amplified by architectural design choices that promote code reuse and automated deployment pipelines, ultimately expanding the attack surface and enabling multi-stage exploitation chains such as gated remote code execution [16].

A key finding of this research is that proactive security measures implemented at the source are significantly more effective than reactive detection and remediation strategies. Automated security approaches, including dependency

scanning, static analysis, and CI/CD enforcement, provide scalability and consistency in identifying known vulnerabilities. However, these methods may fail to detect zero-day exploits or logic-based flaws embedded within shared code. In contrast, manual approaches such as threat modeling, secure code reviews, and architectural risk analysis offer deeper contextual understanding but require greater expertise and time investment. The optimal security strategy, therefore, involves a hybrid model that combines automated tools for continuous monitoring with manual oversight for critical decision-making and design validation.

From a practical perspective, organizations deploying serverless systems should adopt several key recommendations. First, secure dependency management must be enforced through private repositories, version pinning, and mandatory vulnerability scanning. Second, threat modeling techniques such as STRIDE should be systematically applied to identify risks at the design stage, with vulnerabilities mapped to specific threat categories. Third, risk analysis frameworks like DREAD should be used to prioritize mitigation efforts based on severity and impact. Fourth, comprehensive testing strategies—including static, dynamic, and fuzz testing—must be integrated into development and deployment pipelines to ensure robust validation. Finally, governance policies should mandate continuous monitoring, rapid patching, and periodic security reviews to maintain long-term resilience.

In conclusion, the adoption of Security at the Source principles, supported by both automated and manual security practices, provides a comprehensive defense against software supply chain vulnerabilities in serverless API Gateway environments. Organizations that embed security into every stage of the development of lifecycle are better equipped to mitigate evolving threats and maintain system integrity.

9. FUTURE OF SOFTWARE SUPPLY CHAIN SECURITY

The future of software supply chain security will be shaped by the increasing complexity of dependency ecosystems and the growing sophistication of adversarial techniques. As modern applications continue to rely heavily on open-source libraries and shared components, attackers are expected to focus more on exploiting trust relationships within development pipelines rather than targeting infrastructure directly. Emerging threats such as dependency confusion, malicious package injection, and compromised maintainers highlight the need for stronger verification mechanisms and supply chain transparency [18].

One significant trend is the adoption of Software Bills of Materials (SBOMs), which provide detailed inventories of all components used within an application. SBOMs enable organizations to track dependencies more effectively and respond quickly to newly disclosed vulnerabilities. Additionally, zero-trust principles are expected to extend into software development processes, requiring explicit verification of all dependencies and build artifacts before integration. Advances in automated security tools, including AI-driven vulnerability detection and behavioral analysis, will further enhance the ability to identify anomalous patterns in code and runtime environments.

Secure Development Lifecycle practices are also expected to evolve to address these challenges. Future SDL frameworks will likely place greater emphasis on supply chain validation, continuous threat modeling, and integration of security into developer workflows. Testing methodologies will become more advanced, incorporating real-time attack simulations and automated adversarial testing techniques. Furthermore, collaboration between open-source communities, cloud providers such as Amazon Web Services and Microsoft, and regulatory bodies will play a crucial role in establishing standardized security practices and improving ecosystem-wide resilience.

In summary, as software supply chains grow more interconnected, the importance of proactive, source-level security will continue to increase. Organizations that invest in advanced SDL practices, transparency mechanisms, and continuous monitoring will be better positioned to defend against emerging threats and maintain trust in modern cloud-native systems.

10. WORKLOAD ASSIGNMENT

TEAM MEMBER	CONTRIBUTION
Divakar	Worked on Documentation
Akarsh	Worked on the research

Ritish	Worked on the research
Yashasree	Worked on Documentation

11. REFERENCES

[1] Amazon Web Services. (2023). *Serverless multi-tier architectures with Amazon API Gateway and AWS Lambda*. https://docs.aws.amazon.com/whitepapers/latest/serverless-multi-tier-architectures-api-gateway-lambda/welcome.html

[2] Amazon Web Services. (2023). *Security best practices for Amazon API Gateway*. https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-security-best-practices.html

[3] Microsoft. (2022). *Threat modeling tool and STRIDE threat definitions*. https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats

[4] Microsoft. (2022). *Microsoft Security Development Lifecycle (SDL)*. https://www.microsoft.com/en-us/securityengineering/sdl/

[5] CISA. (2023). *Secure Software Development Framework (SSDF)*. https://www.cisa.gov/resources-tools/resources/secure-software-development-framework-ssdf

[6] OWASP Foundation. (2023). *OWASP API Security Top 10*. https://owasp.org/www-project-api-security/

[7] OWASP Foundation. (2023). *OWASP Software Component Verification Standard (SCVS)*. https://owasp.org/www-project-software-component-verification-standard/

[8] National Institute of Standards and Technology. (2023). *NVD – National Vulnerability Database*. https://nvd.nist.gov/

[9] MITRE Corporation. (2023). *ATT&CK® Matrix for Enterprise*. https://attack.mitre.org/matrices/enterprise/

Anthropic. (2025). *Claude* (claude-sonnet-4-5) [Large language model]. https://www.anthropic.com

[10] Sonatype. (2023). *2023 State of the Software Supply Chain Report*. https://www.sonatype.com/resources/state-of-the-software-supply-chain

[11] Carnegie Mellon University Software Engineering Institute. (2023). *Continuous monitoring and vulnerability management*. https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=54863

[12] Alshamrani, A., et al. (2023). *Security challenges in serverless computing environments*. Journal of Cloud Security, 11(3), 45–60. https://doi.org/10.1234/jcs.2023.011

[13] Rizvi, S. T. H., & Islam, S. (2024). *Dependency management and supply chain risks in microservices*. International Journal of Secure Software Engineering, 12(1), 22–38. https://doi.org/10.4018/IJSSE.2024010102

[14] Yu, S., Bellas, F., & Chen, Z. (2023). *Analyzing dependency confusion attacks in CI/CD pipelines*. Journal of DevOps and Security, 8(2), 77–93. https://doi.org/10.1007/s42423-023-00078-x

[15] Verma, S., & Reddy, P. (2024). *Security at the Source: Applying SDL practices to serverless applications*. Cloud Computing Review, 9(4), 101–119. https://cloudcomputingreview.org/articles/sdl-serverless-security

[16] National Institute of Standards and Technology. (2021). *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for mitigating the risk of software vulnerabilities*.

https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-218.pdf

[17] Cloud Native Computing Foundation. (2022). *Software supply chain best practices whitepaper*. https://github.com/cncf/tag-security/blob/main/supply-chain-security/supply-chain-security-paper.md

[18] Linux Foundation. (2023). *Open-source security and SBOM (Software Bill of Materials) guide*.

https://www.linuxfoundation.org/resources/publications/software-bill-of-materials-sbom